

Program 7: Ansible — Complete Explanation from Zero

PART 1: Install WSL and Ubuntu

STEP 1: Enable WSL Feature in Windows

1. Click the Windows **Start button**
 2. Type: `Windows features on or off`
 3. Click on it to open
 4. A list of features appears. Scroll down and find **"Windows Subsystem for Linux"**
 5. Check the checkbox next to it
 6. Click **OK**
 7. Windows will say "Windows needs to restart to apply changes"
 8. Click **"Restart now"**
 9. Wait for computer to restart
-

STEP 2: Install Ubuntu Using Command Prompt

After restart, open **Command Prompt as Administrator**:

1. Click Start
2. Type `cmd`
3. Right click on "Command Prompt"
4. Click **"Run as administrator"**

Now type this to see available Linux distributions:

```
wsl --list --online
```

You will see a list of Linux distributions. We want Ubuntu 24.04.

Type this to install it:

```
wsl --install --distribution Ubuntu-24.04
```

This will download and install Ubuntu. It takes a few minutes depending on your internet speed. You will see:

Downloading: Ubuntu 24.04 LTS
Installing: Ubuntu 24.04 LTS
Distribution successfully installed.

STEP 3: Set Up Ubuntu

After installation, type:

wsl

Ubuntu will start for the first time and ask you to create a user account:

Create a default Unix user account:

Type a username (e.g. `itishree`) and press Enter

New password:

Type a password and press Enter. Note: when typing password in Linux, nothing shows on screen — this is normal, just type and press Enter.

Retype new password:

Type same password again.

You will see a welcome message and your prompt will change to something like:

`itishree@DESKTOP-XXXXX:~$`

This means you are now inside Ubuntu Linux on your Windows computer!

The `$` at the end means you are a regular user. If you see `#` it means you are root (admin).

PART 2: Install Ansible

STEP 4: Update Ubuntu Package List

Inside the Ubuntu terminal type:

```
sudo apt update
```

What does this mean?

- **sudo** — means "do this as administrator" (like right-click → Run as admin in Windows)
- **apt** — the package manager for Ubuntu (like Play Store but for Linux software)
- **update** — refresh the list of available software and their versions

It will ask:

```
[sudo] password for itishree:
```

Type your password and press Enter. You will see a lot of text scrolling — this is normal. Wait for it to finish.

STEP 5: Install Ansible

```
sudo apt install ansible -y
```

What does this mean?

- **install ansible** — download and install Ansible
- **-y** — automatically say "yes" to all questions (so you don't have to type yes repeatedly)

This will take a few minutes. You will see many packages being downloaded and installed.

After it finishes type this to verify Ansible installed correctly:

```
ansible --version
```

You should see something like:

```
ansible [core 2.16.x]
```

This means Ansible is installed successfully!

PART 3: Set Up the Ansible Project

STEP 6: Create a Project Folder

Inside the Ubuntu terminal:

```
mkdir ansible-localhost
```

This creates a new folder called `ansible-localhost`.

```
cd ansible-localhost
```

This goes inside that folder.

Your prompt now shows:

```
itishree@DESKTOP-XXXXX:~/ansible-localhost$
```

STEP 7: Create the Inventory File

The inventory file tells Ansible which machines to manage.

Type:

```
nano hosts.ini
```

What is nano? `nano` is a simple text editor inside Linux terminal. It opens a file for editing directly in the terminal — no separate window opens.

After typing this command, your screen changes to a text editor. You will see the filename at the top and some keyboard shortcuts at the bottom.

Now type this content:

```
[local]
```

localhost ansible_connection=local

What does this mean?

- `[local]` — this is a group name. We are grouping our machines under the name "local"
- `localhost` — the machine we want to manage (localhost means "this same computer")
- `ansible_connection=local` — instead of connecting via SSH (which would be used for remote machines), connect locally since we are managing the same machine

How to save in nano:

1. Press **CTRL + O** (the letter O, not zero) — this means "Write Out" (save)
2. It asks for filename — just press **Enter** to keep the same name
3. Press **CTRL + X** — this exits nano

You are back at the terminal prompt.

STEP 8: Test Ansible Connection with Ping Module

Before running any playbook, let's test if Ansible can connect to our machine.

Type:

```
ansible -i hosts.ini all -m ping
```

What does this command mean?

- `ansible` — run ansible
- `-i hosts.ini` — use `hosts.ini` as the inventory file (`-i` stands for inventory)
- `all` — target all machines listed in the inventory
- `-m ping` — use the ping module (`-m` stands for module)

Expected output:

```
localhost | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
```

What does this output mean?

- **SUCCESS** — Ansible connected successfully
- **"ping": "pong"** — the ping module got a pong response, like network ping/pong
- **"changed": false** — ping doesn't change anything on the machine

If you see **SUCCESS**, Ansible is working correctly!

PART 4: Write and Run a Playbook

STEP 9: Create the index.html File

Our goal is to use Ansible to:

1. Install Nginx web server
2. Replace the default Nginx page with our custom HTML page
3. Start Nginx

First let's create our custom HTML file.

Type:

```
nano index.html
```

Type this content:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Custom Nginx Page</title>
</head>
<body>
  <h1>Hello from Ansible!</h1>
  <p>This is a custom web page deployed using Ansible!</p>
</body>
</html>
```

Save with **CTRL + O**, Enter, **CTRL + X**.

STEP 10: Create the Playbook File

Type:

nano ansible.yml

Type this content carefully. **YAML is very sensitive to spaces** — use exactly 2 spaces for each level of indentation, never use Tab key:

- name: Install and run nginx on localhost

 - hosts: localhost

 - become: yes

 - tasks:

 - name: Install nginx

 - apt:

 - name: nginx

 - state: present

 - update_cache: yes

 - name: Copy custom index.html

 - copy:

 - src: index.html

 - dest: /var/www/html/index.html

 - owner: www-data

 - group: www-data

 - mode: '0644'

 - name: Start nginx service

 - service:

 - name: nginx

 - state: started

 - enabled: yes

Save with **CTRL + O**, Enter, **CTRL + X**.

Understanding the Playbook Line by Line

This is very important for your viva and writeup:

Three dashes mark the beginning of a YAML file. Always start with this.

- name: Install and run nginx on localhost

The `-` means this is a list item. `name` is just a human-readable description of what this play does.

hosts: localhost

Which machine(s) to run this on. `localhost` refers to the group we defined in inventory file.

become: yes

Run all tasks with `sudo` (administrator) privileges. Installing software requires admin rights.

tasks:

Below this comes the list of tasks to perform.

```
- name: Install nginx
  apt:
    name: nginx
    state: present
    update_cache: yes
```

Task 1 — Install nginx.

- `apt` is the module used
 - `name: nginx` — the package to install
 - `state: present` — make sure nginx is present/installed. If already installed, do nothing
 - `update_cache: yes` — run `apt update` before installing (refresh package list)
- ```
- name: Copy custom index.html
 copy:
 src: index.html
```



```
dest: /var/www/html/index.html
owner: www-data
group: www-data
mode: '0644'
```

### Task 2 — Copy our HTML file.

- `copy` is the module used
- `src: index.html` — source file (in our current folder)
- `dest: /var/www/html/index.html` — destination (where Nginx serves files from)
- `owner: www-data` — the file should be owned by www-data user (Nginx user)
- `group: www-data` — same for group
- `mode: '0644'` — file permissions (owner can read/write, others can only read)

```
- name: Start nginx service
 service:
 name: nginx
 state: started
 enabled: yes
```

### Task 3 — Start Nginx.

- `service` is the module used
- `name: nginx` — which service
- `state: started` — make sure it is running
- `enabled: yes` — automatically start nginx when the computer restarts

---

## STEP 11: Verify Your Files

Let's check all our files are in the right place. Type:

```
ls
```

`ls` means "list" — it shows all files in current folder.

You should see:

```
ansible.yml hosts.ini index.html
```

All three files are there. Good.

---

## STEP 12: Run the Playbook

Now the exciting part — run the playbook!

Type:

```
ansible-playbook -i hosts.ini ansible.yml
```

### What does this command mean?

- `ansible-playbook` — command to run a playbook
- `-i hosts.ini` — use hosts.ini as inventory file
- `ansible.yml` — the playbook to run

It will ask for your sudo password:

BECOME password:

Type your Ubuntu password and press Enter.

You will see output like this:

```
PLAY [Install and run nginx on localhost] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Install nginx] *****
changed: [localhost]

TASK [Copy custom index.html] *****
changed: [localhost]

TASK [Start nginx service] *****
changed: [localhost]

PLAY RECAP *****
localhost : ok=4 changed=3 unreachable=0 failed=0
```

### Understanding this output (Important for Viva):

- **PLAY [Install and run nginx on localhost]** — the play started
  - **TASK [Gathering Facts]** — Ansible first collects information about your system (OS version, IP address etc.). This happens automatically before every playbook
  - **ok: [localhost]** — this task ran successfully with no changes
  - **changed: [localhost]** — this task ran and made a change to the system
  - **PLAY RECAP** — summary at the end
  - **ok=4** — 4 tasks ran successfully
  - **changed=3** — 3 tasks made changes
  - **unreachable=0** — no connection failures
  - **failed=0** — no failures. This is the most important one — if this is 0, everything worked!
- 

### STEP 13: Verify in Browser

Now open your Windows browser and go to:

`http://localhost`

You should see a webpage showing:

**Hello from Ansible!** This is a custom web page deployed using Ansible!

**This means Ansible successfully:**

1. Installed Nginx web server
2. Copied your custom HTML file to the right location
3. Started the Nginx service

All of this happened automatically just by running one command!

---

### STEP 14: Run Playbook Again to See Idempotency

Run the same command again:

```
ansible-playbook -i hosts.ini ansible.yml
```

This time you will see:

## PLAY RECAP

localhost : ok=4 changed=0 unreachable=0 failed=0

Notice **changed=0** now! Before it was **changed=3**.

### **This is called Idempotency — Very Important for Viva**

Idempotency means — running the same playbook multiple times gives the same result and does not make unnecessary changes. Since nginx is already installed, the file is already copied and the service is already running — Ansible is smart enough to detect this and do nothing. It does not reinstall or restart unnecessarily.

---

## Complete Summary of All Commands

### — INSTALL WSL AND UBUNTU —

(In Windows search) Windows features on or off

→ Check: Windows Subsystem for Linux → OK → Restart

(Open CMD as Administrator)

wsl --list --online

wsl --install --distribution Ubuntu-24.04

wsl

→ Create username and password

### — INSIDE UBUNTU TERMINAL —

sudo apt update

sudo apt install ansible -y

ansible --version

### — SET UP PROJECT —

mkdir ansible-localhost

cd ansible-localhost

nano hosts.ini

→ type: [local]

→ type: localhost ansible\_connection=localhost

→ CTRL+O, Enter, CTRL+X

ansible -i hosts.ini all -m ping

→ should see: SUCCESS pong

### — CREATE FILES —

nano index.html

- paste HTML content
- CTRL+O, Enter, CTRL+X

nano ansible.yml

- paste playbook content
- CTRL+O, Enter, CTRL+X

ls

- should see: ansible.yml hosts.ini index.html

### — RUN PLAYBOOK —

ansible-playbook -i hosts.ini ansible.yml

- enter password when asked
- should see: failed=0

### — VERIFY —

Open browser → <http://localhost>

- should see: Hello from Ansible!

---

## What You Write in Writeup

**Aim:** Configuration Management with Ansible — Inventory, Playbooks, Modules, Automating Server Configuration

**Steps:**

1. Enable WSL feature in Windows and restart
2. Install Ubuntu 24.04 using `wsl --install --distribution Ubuntu-24.04`
3. Update Ubuntu using `sudo apt update`
4. Install Ansible using `sudo apt install ansible -y`
5. Create project folder `ansible-localhost` and navigate into it
6. Create inventory file `hosts.ini` with localhost configuration
7. Test connection using `ansible -i hosts.ini all -m ping`
8. Create `index.html` with custom content
9. Create playbook `ansible.yml` with tasks to install nginx, copy HTML, start service

10. Run playbook using `ansible-playbook -i hosts.ini ansible.yml`
11. Verify by opening `http://localhost` in browser